# Table of Contents

## 1 Prerequisites

- Kubernetes cluster (kind for local dev, or EKS / GKE / AKS for production)
- kubectl configured with cluster access
- Docker (or Podman) for building container images
- Python 3.11+ with pip (for FastMCP server)
- Node.js 20+ (only if using TypeScript SDK or MCP Inspector)
- Domain with TLS certificate (production only; use port-forward for dev)

**Local dev cluster with kind**

```
# Install kind (macOS)
brew install kind

# Create cluster
kind create cluster --name mcp-demo
```

## 2 Write the MCP Server

**server.py**

```
from mcp.server.fastmcp import FastMCP
import os
import httpx
import requests

mcp = FastMCP("demo-server", host="0.0.0.0", port=8000)

def get_api_key():
    """Helper to ensure we always fetch the latest env variable."""
    return os.getenv("OPENWEATHER_API_KEY")

@mcp.tool()
def check_api_key_status():
    """Check if the OpenWeather API key is loaded."""
    key = get_api_key()
    if key:
        masked = f"{key[:4]}{'*' * (len(key)-8)}{key[-4:]}"
        return {"status": "loaded", "api_key": masked}
    return {"status": "not_loaded", "error": "KEY not set"}

@mcp.tool()
def get_weather(city: str, api_key_override: str = ""):
    """Fetches the current weather for a given city."""
    key_to_use = api_key_override or get_api_key()
    if not key_to_use:
        return {"error": "No API key provided or loaded"}
    base = "http://api.openweathermap.org/data/2.5/weather"
    url = f"{base}?q={city}&appid={key_to_use}&units=metric"
    try:
        response = httpx.get(url, timeout=5.0)
        response.raise_for_status()
        data = response.json()
        return {
            "city": data.get("name", city),
            "temperature": data["main"]["temp"],
            "description": data["weather"][0]["description"]
        }
    except Exception as e:
        return {"error": str(e)}

if __name__ == "__main__":
    mcp.run(transport="streamable-http")
```

**requirements.txt**

```
mcp[cli]>=1.9.0
uvicorn
httpx
requests
```

! FastMCP defaults to host 0.0.0.0 and port 8000 for streamable-http transport. Do not pass host/port in the constructor; they are ignored for this transport.
Align your Kubernetes containerPort to 8000.

## 3 Containerize with Docker

**Dockerfile**

```
FROM python:3.12-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY server.py .

# Security: run as non-root
RUN adduser --disabled-password --no-create-home appuser
USER appuser

EXPOSE 8000
CMD ["python", "server.py"]
```

**Build and load into kind**

```
# Build the image
docker build -t demo-server:latest .

# Load into kind cluster (NOT docker push!)
kind load docker-image demo-server:latest --name mcp-demo
```

> ! kind runs Kubernetes inside Docker containers with an isolated image registry. You must use 'kind load docker-image' to make images available. Set imagePullPolicy: Never in the deployment, otherwise K8s will try to pull from Docker Hub and fail with ErrImagePull.

## 4 Kubernetes Manifests

### 4a — Namespace

```
apiVersion: v1
kind: Namespace
metadata:
  name: mcp-servers
```

### 4b — ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: weather-service-config
data:
  DEFAULT_CITY: "Katowice"
  UNITS: "metric"
  API_URL: "http://api.openweathermap.org/data/2.5/weather"
```

✓ ConfigMaps store non-sensitive configuration as key-value pairs. Pods reference them via envFrom or individual env valueFrom entries. For sensitive data like API keys, use Secrets instead.

### 4c — Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mcp-demo-server
  namespace: mcp-servers
spec:
  replicas: 2
  selector:
    matchLabels:
      app: mcp-demo-server
  template:
    metadata:
      labels:
        app: mcp-demo-server
    spec:
      containers:
        - name: mcp-server
          image: demo-server:latest
          imagePullPolicy: Never
          ports:
            - containerPort: 8000
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
      securityContext:
        runAsNonRoot: true
        runAsUser: 1000
```

> ! Do NOT add liveness/readiness probes pointing to /health unless your MCP server explicitly implements that endpoint. FastMCP does not expose /health by default, and a 404 response will cause Kubernetes to restart pods in a CrashLoopBackOff cycle.

### 4d — Service

```
apiVersion: v1
kind: Service
metadata:
  name: mcp-demo-server
  namespace: mcp-servers
spec:
  selector:
    app: mcp-demo-server
  ports:
    - port: 80
      targetPort: 8000
  type: ClusterIP
```

### 4e — Ingress with TLS (production)

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: mcp-demo-server
  namespace: mcp-servers
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - mcp.your-domain.com
      secretName: mcp-tls
  rules:
    - host: mcp.your-domain.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: mcp-demo-server
                port:
                  number: 80
```

## 5 Deploy to Cluster

```
# Apply all manifests
kubectl apply -f k8s/

# Verify pods are running
kubectl -n mcp-servers get pods

# Check logs
kubectl -n mcp-servers logs -l app=mcp-demo-server --tail=20

# Expected log output:
# INFO:  Uvicorn running on http://0.0.0.0:8000
```

You should see pods in **Running 1/1** status. If you see **ErrImagePull** or **ErrImageNeverPull**, revisit Step 5 (image loading). If you see **CrashLoopBackOff**, check the logs for Python errors or health probe issues.

## 6 Validate & Test

**Port-forward to local machine**

```
kubectl -n mcp-servers port-forward svc/mcp-demo-server 8080:80
```

**Test with curl**

```
curl -X POST http://localhost:8080/mcp \
  -H "Content-Type: application/json" \
  -H "Accept: application/json, text/event-stream" \
  -d '{
    "jsonrpc": "2.0", "id": 1,
    "method": "initialize",
    "params": {
      "protocolVersion": "2025-03-26",
      "clientInfo": {"name": "test", "version": "1.0"},
      "capabilities": {}
    }
  }'
```

> ! The Accept header must include both application/json and text/event-stream.
> Without it, the server returns a 'Not Acceptable' error.

**List available tools**

```
curl -X POST http://localhost:8080/mcp \
  -H "Content-Type: application/json" \
  -H "Accept: application/json, text/event-stream" \
  -d '{"jsonrpc":"2.0","id":2,"method":"tools/list","params":{}}'
```

**Test with MCP Inspector**

```
npx @modelcontextprotocol/inspector

# Transport: Streamable HTTP
# URL: http://localhost:8080/mcp
# Copy the session token from terminal into Configuration
```

## 7 Connect AI Clients

**Claude Desktop / Claude Code**

```
// ~/.claude/claude_code_config.json
{
  "mcpServers": {
    "demo-server": {
      "url": "https://mcp.your-domain.com/mcp"
    }
  }
}
```

**Cursor / VS Code**

```
// .cursor/mcp.json  or  .vscode/mcp.json
{
  "servers": {
    "demo-server": {
      "url": "https://mcp.your-domain.com/mcp"
    }
  }
}
```

For local development with port-forward, replace the URL with **http://localhost:8080/mcp**.

## 8 Production Security Checklist

- **TLS Everywhere** — Use cert-manager with Ingress or a service mesh (Istio/Linkerd).
- **Authentication** — Place OAuth2 Proxy in front of Ingress or use bearer tokens in MCP auth headers.
- **RBAC** — Create a dedicated ServiceAccount with minimal permissions if the server interacts with K8s API.
- **Network Policies** — Restrict ingress to the Ingress controller namespace; restrict egress to required APIs only.
- **Secrets Management** — Use External Secrets Operator or Sealed Secrets instead of plain K8s Secrets.
- **Rate Limiting** — Add nginx.ingress.kubernetes.io/limit-rps annotation to prevent abuse.
- **Read-Only Mode** — Run the server in read-only mode by default; enable writes only when necessary.
- **Non-Root Container** — Always set runAsNonRoot: true and a specific runAsUser in the security context.

**Example: Network Policy**

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: mcp-server-policy
  namespace: mcp-servers
spec:
  podSelector:
    matchLabels:
      app: mcp-demo-server
  policyTypes: [Ingress, Egress]
  ingress:
    - from:
        - namespaceSelector:
            matchLabels: { name: ingress-nginx }
      ports: [{ port: 8000 }]
  egress:
    - to:
        - ipBlock: { cidr: 0.0.0.0/0 }
      ports: [{ port: 443 }]
```

## 9 Quick Reference

### Complete deployment flow

```
# 1. Write server
vim server.py                      # FastMCP + @mcp.tool()

# 2. Build container
docker build -t demo-server:latest .

# 3. Load into kind
kind load docker-image demo-server:latest --name mcp-demo

# 4. Deploy
kubectl apply -f k8s/

# 5. Verify
kubectl -n mcp-servers get pods
kubectl -n mcp-servers logs -l app=mcp-demo-server --tail=20

# 6. Test
kubectl -n mcp-servers port-forward svc/mcp-demo-server 8080:80
curl -X POST http://localhost:8080/mcp \
  -H "Content-Type: application/json" \
  -H "Accept: application/json, text/event-stream" \
  -d '{"jsonrpc":"2.0","id":1,"method":"tools/list","params":{}}'

# 7. Connect AI client
# Add URL to claude_code_config.json / .cursor/mcp.json
```

### Useful diagnostic commands

```
kubectl -n mcp-servers describe pod <pod-name>       # Detailed status
kubectl -n mcp-servers get events --sort-by=.lastTimestamp
kubectl -n mcp-servers exec -it <pod> -- sh           # Shell into pod
kind get clusters                                     # List kind clusters
docker exec kind-control-plane crictl images | grep mcp  # Verify image
```

### Alternative tooling

- **kmcp (kagent)** — CLI for scaffolding, building, and deploying MCP servers on K8s with MCPServer CRDs.
- **ToolHive (Stacklok)** — K8s operator with proxy, per-server RBAC, and multi-tenant support.
- **Helm chart** — helm install kubernetes-mcp-server oci://ghcr.io/containers/charts/kubernetes-mcp-server
- **Cyclops** — UI-based deployment of MCP servers on K8s with custom templates.